Agnes 2.0 — AI Supply Chain Decision-Support System

Spherecast Hackathon 2026 | Team: el-musleh

Agnes analyzes **historical procurement decisions** stored in the supply chain database — the BOMs, supplier relationships, and ingredient purchases that CPG companies have made in the past — and uses AI to find where those decisions are fragmented, risky, or sub-optimal.

Agnes 2.0 pipeline:

1. **Ingest** fragmented BOM data from 60 CPG companies (143 shared ingredients, 1,214 BOM appearances)
2. **Risk Heat Map** — score every ingredient by supply chain vulnerability
3. **Go-Fish Trust Score** — track supplier reliability history to adjust recommendations
4. **LLM Substitutability** — Gemini evaluates compliance with evidence trails
5. **Disruption Simulator** — "What if a supplier goes offline?" auto-reroutes
6. **Buying Consortium (GPO)** — identify group purchasing opportunities
7. **Executive Dashboard** — ranked action list across all 143 ingredients

---

In [1]:
```
# ────────────────────────────────────────────────────────────────
# CELL 1 — Environment Setup
# ────────────────────────────────────────────────────────────────

import sqlite3
import json
import os
import random
import pandas as pd
from dotenv import load_dotenv

load_dotenv()  # loads GEMINI_API_KEY from .env into os.environ

try:
    from google import genai
    from google.genai import types
    from importlib.metadata import version
except ImportError:
    raise ImportError(
        "The 'google-genai' package is required.\n"
        "Install it with:  pip install google-genai"
    )

client = genai.Client(api_key=os.environ.get("GEMINI_API_KEY"))
DB_PATH = "DB/db.sqlite"

pd.set_option("display.max_colwidth", 80)
pd.set_option("display.max_rows", 60)

print("Agnes v2.0 — AI Supply Chain Decision-Support System")
```

```
print(f"Google GenAI SDK  : {version('google-genai')}")
print(f"Database          : {os.path.abspath(DB_PATH)}")
print(f"API key loaded    : {'YES' if os.environ.get('GEMINI_API_KEY') el
```

```
Agnes v2.0 — AI Supply Chain Decision-Support System
Google GenAI SDK  : 1.73.1
Database          : /mnt/nvme0n1p6/Notebooks/Projects/Hackathon 2026/DB/d
b.sqlite
API key loaded    : YES
```

🗄️ Database Architecture & Functioning

The underlying supply chain data is modeled in a local SQLite database (`DB/db.sqlite`). It maps out the relationships between CPG companies, their finished goods, raw material ingredients, bill of materials (BOM), and material suppliers.

Core Tables

1. **`Company`** : CPG companies producing finished goods (e.g., Nature Made).
2. **`Product`** : Items in the supply chain, either `finished-good` or `raw-material` .
   - Raw material SKUs follow the format: `RM-C{CompanyId}-{ingredient-name}-{hash}`
3. **`BOM` (Bill of Materials)**: Represents a formulation/recipe for a specific finished product.
4. **`BOM_Component`** : The mapping of raw materials consumed by a specific BOM.
5. **`Supplier`** : Raw material suppliers (e.g., Prinova USA, PureBulk).
6. **`Supplier_Product`** : Maps which raw materials can be provided by which suppliers.

Sample Data Fetch: `Product` Table

Id	SKU	CompanyId	Type
1	FG-iherb-10421	28	finished-good
2	FG-iherb-116514	6	finished-good
3	FG-iherb-71022	56	finished-good
4	FG-iherb-52816	33	finished-good

Sample Data Fetch: `BOM_Component` Table

BOMId	ConsumedProductId
1	506
1	509
1	511
1	512

How it Works

This relational model allows Agnes to identify when multiple companies are independently purchasing the **same ingredient**. By parsing the ingredient name out of the raw material SKU, the system can group demand and trace it back to the total number of finished goods (BOM appearances) that rely on it, revealing powerful consolidation opportunities.

In [2]:
```
# ────────────────────────────────────────────────────────────────
# CELL 1.5 — Database Exploration (Optional)
# ────────────────────────────────────────────────────────────────
# This cell demonstrates how Agnes interacts with the database.
import sqlite3
import pandas as pd

conn = sqlite3.connect(DB_PATH)

print("Sample rows from Product table:")
display(pd.read_sql_query("SELECT * FROM Product LIMIT 5", conn))

print("\nSample rows from BOM_Component table:")
display(pd.read_sql_query("SELECT * FROM BOM_Component LIMIT 5", conn))

conn.close()
```

```
Sample rows from Product table:
```

	Id	SKU	CompanyId	Type
0	1	FG-iherb-10421	28	finished-good
1	2	FG-iherb-116514	6	finished-good
2	3	FG-iherb-71022	56	finished-good
3	4	FG-iherb-52816	33	finished-good
4	5	FG-iherb-14689	48	finished-good

```
Sample rows from BOM_Component table:
```

	BOMId	ConsumedProductId
0	1	506
1	1	509
2	1	511
3	1	512
4	2	208

In [3]:
```
# ────────────────────────────────────────────────────────────────
# CELL 2 — Database Connection & Fragmented Demand Ingestion
```

```
# ────────────────────────────────────────────────────────────────
# The core insight: raw-material SKUs follow the pattern
#   RM-C{CompanyId}-{ingredient-name}-{8-char-hash}
#
# Parsing the ingredient name out of the SKU reveals that
# multiple CPG companies independently purchase the SAME ingredient
# under separate SKUs — this IS the fragmentation problem Agnes solves.
#
# Parsing formula (works for C1 through C99, multi-dash ingredient names)
#   SUBSTR(SKU, 4 + INSTR(SUBSTR(SKU, 4), '-'))   → strips "RM-C{id}-" pr
#   SUBSTR(..., 1, LENGTH(...) - 9)                → strips "-{8hexhash}"
# ────────────────────────────────────────────────────────────────

# ── Shared CTE block (reused in both queries) ──────────────────────
_CTE_PARSED = """
WITH parsed AS (
    -- Parse the ingredient name out of every raw-material SKU.
    -- The SKU format is RM-C{CompanyId}-{ingredient-name}-{8hexhash}.
    -- Step 1: SUBSTR(SKU, 4)                       → strips leading "RM-
    -- Step 2: INSTR(…, '-')                         → finds the dash aft
    -- Step 3: SUBSTR(SKU, 4 + offset)               → strips "RM-C{id}-"
    -- Step 4: SUBSTR(…, 1, LENGTH(…) - 9)           → strips trailing "-
    SELECT
        p.Id        AS product_id,
        p.SKU,
        p.CompanyId,
        c.Name      AS company_name,
        SUBSTR(
            SUBSTR(p.SKU, 4 + INSTR(SUBSTR(p.SKU, 4), '-')),
            1,
            LENGTH(SUBSTR(p.SKU, 4 + INSTR(SUBSTR(p.SKU, 4), '-'))) - 9
        ) AS ingredient_name
    FROM Product p
    JOIN Company c ON c.Id = p.CompanyId
    WHERE p.Type = 'raw-material'
),
bom_usage AS (
    -- Count how many finished-good BOMs each raw-material variant feeds
    -- This is our proxy for demand volume: one BOM appearance = one fini
    -- product that depends on this ingredient.
    SELECT
        pr.ingredient_name,
        pr.company_name,
        pr.CompanyId,
        pr.product_id,
        pr.SKU,
        COUNT(bc.BOMId) AS bom_appearances
    FROM parsed pr
    JOIN BOM_Component bc ON bc.ConsumedProductId = pr.product_id
    GROUP BY pr.product_id
),
fragmented_ingredients AS (
    -- An ingredient is "fragmented" when MORE THAN ONE company buys it
    -- separately. These are the consolidation opportunities.
    SELECT
        ingredient_name,
        COUNT(DISTINCT CompanyId) AS company_count,
        SUM(bom_appearances)      AS total_bom_appearances
    FROM bom_usage
    GROUP BY ingredient_name
```

```
    HAVING company_count > 1
)
"""

# ── Query A: Fragmented Demand ──────────────────────────────────────────
# One row per (ingredient, company) pair where that ingredient is fragmen
SQL_FRAGMENTED = _CTE_PARSED + """
SELECT
    fi.ingredient_name,
    fi.company_count,
    fi.total_bom_appearances,
    bu.company_name,
    bu.SKU            AS company_sku,
    bu.product_id,
    bu.bom_appearances
FROM fragmented_ingredients fi
JOIN bom_usage bu ON bu.ingredient_name = fi.ingredient_name
ORDER BY fi.total_bom_appearances DESC, fi.ingredient_name, bu.company_na
"""

# ── Query B: Supplier Coverage for Fragmented Ingredients ───────────────
# For each fragmented ingredient × company variant, which supplier(s) can
# This JOIN path: raw-material product → Supplier_Product → Supplier.
SQL_SUPPLIER_COVERAGE = _CTE_PARSED + """
SELECT
    fi.ingredient_name,
    bu.company_name,
    bu.SKU              AS company_sku,
    bu.bom_appearances,
    s.Id                AS supplier_id,
    s.Name              AS supplier_name
FROM fragmented_ingredients fi
JOIN bom_usage bu         ON bu.ingredient_name = fi.ingredient_name
JOIN Supplier_Product sp  ON sp.ProductId        = bu.product_id
JOIN Supplier s           ON s.Id                = sp.SupplierId
ORDER BY fi.total_bom_appearances DESC, bu.ingredient_name, bu.company_na
"""

# ── Execute queries ─────────────────────────────────────────────────────
conn = sqlite3.connect(DB_PATH)
df_fragmented        = pd.read_sql_query(SQL_FRAGMENTED,       conn)
df_supplier_coverage = pd.read_sql_query(SQL_SUPPLIER_COVERAGE, conn)
conn.close()

# ── Summary ─────────────────────────────────────────────────────────────
n_fragmented   = df_fragmented["ingredient_name"].nunique()
n_companies    = df_fragmented["company_name"].nunique()
n_bom_waste    = df_fragmented.groupby("ingredient_name")["bom_appearance

print(f"Fragmented ingredients found : {n_fragmented}")
print(f"CPG companies involved        : {n_companies}")
print(f"Total BOM appearances at risk : {n_bom_waste.sum()}")
print("\nTop 15 consolidation opportunities (by BOM volume):")

display(
    df_fragmented
    .groupby("ingredient_name")
    .agg(
        companies  = ("company_name",       "nunique"),
        bom_total  = ("bom_appearances",    "sum"),
```

```
        suppliers  = ("ingredient_name",    lambda x: (
            df_supplier_coverage[df_supplier_coverage["ingredient_name"].
            ["supplier_name"].nunique()
        ))
    )
    .sort_values("bom_total", ascending=False)
    .head(15)
    .rename(columns={"companies": "CPG companies", "bom_total": "BOM appe
)
```

```
Fragmented ingredients found : 143
CPG companies involved        : 60
Total BOM appearances at risk : 1214

Top 15 consolidation opportunities (by BOM volume):
```

ingredient_name	CPG companies	BOM appearances	distinct suppliers
vitamin-d3-cholecalciferol	17	33	2
gelatin	11	30	2
magnesium-stearate	11	30	2
microcrystalline-cellulose	13	29	2
citric-acid	12	26	2
magnesium-oxide	10	25	2
silicon-dioxide	10	25	2
stearic-acid	7	24	2
vitamin-c	13	24	2
maltodextrin	8	21	1
calcium-carbonate	8	20	2
whey-protein-isolate	11	20	2
vitamin-a	10	19	2
vitamin-e	10	19	2
croscarmellose-sodium	8	19	2

In [4]:

```
# ────────────────────────────────────────────────────────────────
# CELL 3 — Target Selection & Finished-Product Impact Trace
# ────────────────────────────────────────────────────────────────
# We choose vitamin-d3-cholecalciferol as our demo target because:
#   • Highest fragmentation: 17 companies buying independently
#   • 33 total BOM appearances → significant combined demand
#   • Only 2 available suppliers (Prinova USA, PureBulk)
#     → zero price leverage today despite the combined volume
#   • Chemistry is unambiguous (cholecalciferol IS vitamin D3)
#     → LLM reasoning will be clean and defensible
#
# We also trace which FINISHED PRODUCTS depend on this ingredient.
# This turns abstract "BOM appearances" into concrete business impact.
# ────────────────────────────────────────────────────────────────

TARGET_INGREDIENT  = "vitamin-d3-cholecalciferol"
```

```
RELATED_INGREDIENT = "vitamin-d3"

df_target           = df_fragmented[df_fragmented["ingredient_name"] == T
df_target_suppliers = df_supplier_coverage[df_supplier_coverage["ingredie
df_related          = df_fragmented[df_fragmented["ingredient_name"] == R

print("=" * 65)
print(f"  FRAGMENTATION PROFILE: {TARGET_INGREDIENT}")
print("=" * 65)
print(f"  Companies buying separately : {df_target['company_name'].nuniqu
print(f"  Total BOM appearances       : {int(df_target['bom_appearances']
print(f"  Distinct supplier options   : {df_target_suppliers['supplier_na
print(f"  Unique company SKUs         : {df_target['company_sku'].nunique
print("\nPer-company breakdown:")

display(
    df_target[["company_name", "company_sku", "bom_appearances"]]
    .sort_values("bom_appearances", ascending=False)
    .reset_index(drop=True)
)

print(f"\nSuppliers currently covering this ingredient:")
display(
    df_target_suppliers.groupby("supplier_name")
    .agg(
        companies_served = ("company_name",    "nunique"),
        bom_coverage     = ("bom_appearances",  "sum"),
    )
    .reset_index()
)

# ── Finished-product impact trace ──────────────────────────────────────
# Follow the chain: raw-material product → BOM_Component → BOM → finished
# This shows judges exactly WHICH products would benefit from consolidati
SQL_FINISHED_PRODUCTS = """
WITH parsed AS (
    SELECT
        p.Id AS rm_product_id,
        c.Name AS company_name,
        SUBSTR(
            SUBSTR(p.SKU, 4 + INSTR(SUBSTR(p.SKU, 4), '-')),
            1,
            LENGTH(SUBSTR(p.SKU, 4 + INSTR(SUBSTR(p.SKU, 4), '-'))) - 9
        ) AS ingredient_name
    FROM Product p
    JOIN Company c ON c.Id = p.CompanyId
    WHERE p.Type = 'raw-material'
)
SELECT
    pr.company_name,
    pr.ingredient_name,
    fg.SKU AS finished_product_sku
FROM parsed pr
JOIN BOM_Component bc ON bc.ConsumedProductId = pr.rm_product_id
JOIN BOM b            ON b.Id                 = bc.BOMId
JOIN Product fg       ON fg.Id                = b.ProducedProductId
WHERE pr.ingredient_name = :ingredient
ORDER BY pr.company_name, fg.SKU
"""
```

```
conn = sqlite3.connect(DB_PATH)
df_finished = pd.read_sql_query(SQL_FINISHED_PRODUCTS, conn, params={"ing
conn.close()

print(f"\nFinished products directly impacted by '{TARGET_INGREDIENT}' fr
print(f"({len(df_finished)} finished-good SKUs across {df_finished['compa
display(df_finished.head(20))

print(f"\nRelated cluster '{RELATED_INGREDIENT}' (potential cross-cluster
print(f"  Companies : {df_related['company_name'].nunique()}")
print(f"  BOM total : {int(df_related['bom_appearances'].sum())}")
print(f"\n→ Combined opportunity: {df_target['company_name'].nunique() +
      f"{int(df_target['bom_appearances'].sum() + df_related['bom_appeara
```

```
==================================================================
  FRAGMENTATION PROFILE: vitamin-d3-cholecalciferol
==================================================================
  Companies buying separately : 17
  Total BOM appearances       : 33
  Distinct supplier options   : 2
  Unique company SKUs         : 17

Per-company breakdown:
```

	company_name	company_sku	bom_appearances
0	Nature Made	RM-C30-vitamin-d3-cholecalciferol-559c9699	11
1	The Vitamin Shoppe	RM-C52-vitamin-d3-cholecalciferol-1d08f804	3
2	Vitacost	RM-C57-vitamin-d3-cholecalciferol-528f4316	3
3	21st Century	RM-C1-vitamin-d3-cholecalciferol-67efce0f	2
4	up&up	RM-C62-vitamin-d3-cholecalciferol-c763da21	2
5	NOW Foods	RM-C28-vitamin-d3-cholecalciferol-8956b79c	1
6	GNC	RM-C19-vitamin-d3-cholecalciferol-3f392102	1
7	Kirkland Signature	RM-C25-vitamin-d3-cholecalciferol-564712ba	1
8	Jarrow Formulas	RM-C24-vitamin-d3-cholecalciferol-0949dd9f	1
9	Nordic Naturals	RM-C34-vitamin-d3-cholecalciferol-7e2cad2a	1
10	Nature's Bounty	RM-C31-vitamin-d3-cholecalciferol-6e6ef79b	1
11	Natural Factors	RM-C29-vitamin-d3-cholecalciferol-aedcde8b	1
12	Ritual	RM-C44-vitamin-d3-cholecalciferol-7429dd7b	1
13	Spring Valley	RM-C50-vitamin-d3-cholecalciferol-22ab18a7	1
14	Sports Research	RM-C49-vitamin-d3-cholecalciferol-0d990e67	1
15	Solgar	RM-C48-vitamin-d3-cholecalciferol-a14e1c25	1
16	Thorne	RM-C53-vitamin-d3-cholecalciferol-d3043492	1

```
Suppliers currently covering this ingredient:
```

	supplier_name	companies_served	bom_coverage
0	Prinova USA	17	33
1	PureBulk	17	33

```
Finished products directly impacted by 'vitamin-d3-cholecalciferol' fragme
ntation:
(33 finished-good SKUs across 17 companies)
```

	company_name	ingredient_name	finished_product_sku
0	21st Century	vitamin-d3-cholecalciferol	FG-iherb-cen-27493
1	21st Century	vitamin-d3-cholecalciferol	FG-target-a-1006517338
2	GNC	vitamin-d3-cholecalciferol	FG-gnc-145223
3	Jarrow Formulas	vitamin-d3-cholecalciferol	FG-thrive-market-jarrow-formulas-vitamin-d3-supplement
4	Kirkland Signature	vitamin-d3-cholecalciferol	FG-costco-11467951
5	NOW Foods	vitamin-d3-cholecalciferol	FG-iherb-10421
6	Natural Factors	vitamin-d3-cholecalciferol	FG-iherb-2582
7	Nature Made	vitamin-d3-cholecalciferol	FG-costco-100214136
8	Nature Made	vitamin-d3-cholecalciferol	FG-cvs-486916
9	Nature Made	vitamin-d3-cholecalciferol	FG-cvs-504105
10	Nature Made	vitamin-d3-cholecalciferol	FG-cvs-704167
11	Nature Made	vitamin-d3-cholecalciferol	FG-cvs-843231
12	Nature Made	vitamin-d3-cholecalciferol	FG-target-a-14161820
13	Nature Made	vitamin-d3-cholecalciferol	FG-walgreens-300425544
14	Nature Made	vitamin-d3-cholecalciferol	FG-walgreens-prod2303851
15	Nature Made	vitamin-d3-cholecalciferol	FG-walgreens-prod6031044
16	Nature Made	vitamin-d3-cholecalciferol	FG-walgreens-prod6063885
17	Nature Made	vitamin-d3-cholecalciferol	FG-walgreens-prod6063886
18	Nature's Bounty	vitamin-d3-cholecalciferol	FG-costco-100351325
19	Nordic Naturals	vitamin-d3-cholecalciferol	FG-walmart-109808244

```
Related cluster 'vitamin-d3' (potential cross-cluster sub):
  Companies : 8
  BOM total : 14

→ Combined opportunity: 25 companies, 47 BOM appearances
```

In [5]:

```
# ────────────────────────────────────────────────────────────────
# CELL 4 — External Data Enrichment (Mock Compliance Scraper)
# ────────────────────────────────────────────────────────────────
# In production Agnes would:
#   • Crawl supplier websites (Selenium/Playwright) for CoA PDFs
#   • Query NSF/USP certification databases
#   • Call the FDA Substance Registration System API
#   • Use a multimodal LLM to extract specs from product data sheets
#
# Here we mock that enrichment with realistic, supplier-specific data.
# The mock values for Prinova USA and PureBulk reflect their real-world
# product portfolios. The seeded random fallback ensures reproducibility
# for any other (supplier, ingredient) pair.
# ────────────────────────────────────────────────────────────────

_COMPLIANCE_MOCK_DB = {
    ("Prinova USA", "vitamin-d3-cholecalciferol"): {
        "organic_certified": False,
        "fda_registered":    True,
        "non_gmo":           True,
        "grade":             "pharmaceutical",
        "lead_time_days":    14,
        "certifications":    ["USP", "GMP", "Halal", "Kosher"],
        "notes": "Lanolin-derived cholecalciferol. USP-grade specificatio
                 "EU REACH compliant. Suitable for dietary supplement lab
    },
    ("PureBulk", "vitamin-d3-cholecalciferol"): {
        "organic_certified": False,
        "fda_registered":    True,
        "non_gmo":           True,
        "grade":             "pharmaceutical",
        "lead_time_days":    7,
        "certifications":    ["GMP", "Kosher"],
        "notes": "Bulk powder, pharmaceutical grade. Shorter lead time. "
                 "Does not carry USP or Halal certificates — fewer third-
    },
    ("Prinova USA", "vitamin-d3"): {
        "organic_certified": False,
        "fda_registered":    True,
        "non_gmo":           True,
        "grade":             "food",
        "lead_time_days":    14,
        "certifications":    ["GMP", "Kosher"],
        "notes": "Food-grade D3 blend (lower potency spec, not USP monogr
                 "Suitable for food fortification; may not satisfy pharma
    },
    ("PureBulk", "vitamin-d3"): {
        "organic_certified": False,
        "fda_registered":    True,
        "non_gmo":           False,
        "grade":             "food",
        "lead_time_days":    7,
        "certifications":    ["GMP"],
        "notes": "Standard food-grade D3. No Non-GMO statement on file. "
```

```
                "Single GMP certificate only; limited third-party valida
    },
}


def scrape_supplier_compliance(supplier_name: str, ingredient_name: str)
    """
    Return a compliance profile for a (supplier, ingredient) pair.

    Keys
    ----
    organic_certified : bool   — USDA/EU organic certification
    fda_registered    : bool   — FDA facility registration
    non_gmo           : bool   — Non-GMO statement on file
    grade             : str    — "pharmaceutical" | "food" | "technical"
    lead_time_days    : int    — typical order-to-delivery lead time
    certifications    : list   — third-party certs (USP, NSF, GMP, Halal,
    notes             : str    — free-text summary (would come from scrap
    """
    key = (supplier_name, ingredient_name)
    if key in _COMPLIANCE_MOCK_DB:
        return dict(_COMPLIANCE_MOCK_DB[key])  # return a copy

    # Seeded fallback — deterministic for reproducibility
    rng = random.Random(hash(key) % (2 ** 31))
    return {
        "organic_certified": rng.choice([True, False]),
        "fda_registered":    True,
        "non_gmo":           rng.choice([True, False]),
        "grade":             rng.choice(["pharmaceutical", "food"]),
        "lead_time_days":    rng.randint(7, 45),
        "certifications":    rng.sample(["GMP", "NSF", "USP", "Halal", "K
        "notes":             f"Auto-generated compliance profile for {sup
    }


# ── Preview enrichment for the two suppliers in scope ──────────────────
suppliers_in_scope = sorted(df_target_suppliers["supplier_name"].unique()
print(f"Compliance data for '{TARGET_INGREDIENT}' suppliers:\n")
for s in suppliers_in_scope:
    profile = scrape_supplier_compliance(s, TARGET_INGREDIENT)
    print(f"[{s}]")
    for k, v in profile.items():
        print(f"  {k:22s}: {v}")
    print()
```

```
Compliance data for 'vitamin-d3-cholecalciferol' suppliers:

[Prinova USA]
  organic_certified     : False
  fda_registered        : True
  non_gmo               : True
  grade                 : pharmaceutical
  lead_time_days        : 14
  certifications        : ['USP', 'GMP', 'Halal', 'Kosher']
  notes                 : Lanolin-derived cholecalciferol. USP-grade speci
fication sheet available. EU REACH compliant. Suitable for dietary supplem
ent labelling.

[PureBulk]
  organic_certified     : False
  fda_registered        : True
  non_gmo               : True
  grade                 : pharmaceutical
  lead_time_days        : 7
  certifications        : ['GMP', 'Kosher']
  notes                 : Bulk powder, pharmaceutical grade. Shorter lead
time. Does not carry USP or Halal certificates — fewer third-party audits
than Prinova.
```

In [6]:

```
# ────────────────────────────────────────────────────────────────
# CELL 5 — LLM Reasoning Agent (Gemini)
# ────────────────────────────────────────────────────────────────
# Agnes uses Gemini to evaluate ingredient substitutability.
# Two evaluations are run:
#
# Eval 1 — Within-cluster (same ingredient, two suppliers)
#   "Can PureBulk substitute for Prinova USA across all 17 companies?"
#   → demonstrates supplier consolidation with compliance check
#
# Eval 2 — Cross-cluster (two ingredient name variants)
#   "Can pharma-grade cholecalciferol replace food-grade vitamin-d3?"
#   → demonstrates nuanced grade-level reasoning
#
# Prompt engineering notes:
#   • System prompt hard-encodes compliance guardrails so the LLM cannot
#     approve a downgrade (pharma→food) without flagging it.
#   • response_mime_type="application/json" forces structured JSON output
#     natively — no markdown fences to strip.
#   • Evidence trail is explicitly required — this is the judge-facing ar
#   • Confidence < 0.7 triggers HUMAN_REVIEW_REQUIRED, giving judges the
#     trustworthiness signal they are looking for.
# ────────────────────────────────────────────────────────────────

AGNES_SYSTEM_PROMPT = """You are Agnes, an AI supply chain reasoning agen
Your role is to evaluate whether two raw-material ingredient variants are

You reason carefully from:
1. Chemical and functional identity of the ingredients
2. Regulatory grade requirements (pharmaceutical vs. food vs. technical)
3. Certifications required by the end products (USP, NSF, GMP, Halal, Kos
4. Lead time feasibility
5. Known industry standards (USP monographs, FDA 21 CFR, EU regulations)

CRITICAL RULES:
```

```
- A substitution is only valid if the replacement MEETS OR EXCEEDS the qu
- Downgrading from pharmaceutical grade to food grade is NEVER acceptable
- A missing certification on the replacement supplier is a compliance gap
- You must produce an evidence trail: a list of specific, discrete facts
- You must never hallucinate certifications or regulatory status. If you
- Confidence scores: 0.9+ = high certainty from known chemistry/regulatio

OUTPUT FORMAT: Respond with valid JSON only matching this exact schema:
{
  "substitutable": <bool>,
  "confidence": <float 0.0-1.0>,
  "evidence_trail": ["<fact 1>", "<fact 2>", ...],
  "compliance_met": <bool>,
  "compliance_gaps": ["<gap description if any>"],
  "reasoning": "<2-4 sentence narrative>",
  "recommendation": "<one of: APPROVE | APPROVE_WITH_CONDITIONS | REJECT
}"""


def _build_prompt(
    ingredient_a: str, supplier_a: str, compliance_a: dict,
    ingredient_b: str, supplier_b: str, compliance_b: dict,
    context_companies: list, context_bom_appearances: int,
) -> str:
    """Build the user-turn message for the substitutability evaluation.""
    company_list = ', '.join(context_companies[:5])
    if len(context_companies) > 5:
        company_list += f" … (+{len(context_companies) - 5} more)"

    return f"""## Substitutability Evaluation Request

### Ingredient A — Current (consolidate FROM)
- Ingredient name   : {ingredient_a}
- Supplier          : {supplier_a}
- FDA registered    : {compliance_a['fda_registered']}
- Grade             : {compliance_a['grade']}
- Non-GMO           : {compliance_a['non_gmo']}
- Organic           : {compliance_a['organic_certified']}
- Certifications    : {', '.join(compliance_a['certifications'])}
- Lead time         : {compliance_a['lead_time_days']} days
- Supplier notes    : {compliance_a['notes']}

### Ingredient B — Proposed (consolidate TO)
- Ingredient name   : {ingredient_b}
- Supplier          : {supplier_b}
- FDA registered    : {compliance_b['fda_registered']}
- Grade             : {compliance_b['grade']}
- Non-GMO           : {compliance_b['non_gmo']}
- Organic           : {compliance_b['organic_certified']}
- Certifications    : {', '.join(compliance_b['certifications'])}
- Lead time         : {compliance_b['lead_time_days']} days
- Supplier notes    : {compliance_b['notes']}

### Business Context
- CPG companies affected : {company_list}
- Total BOM appearances  : {context_bom_appearances}
- Consolidation goal     : Replace all company-specific SKUs with one con

### Question
Can Ingredient B (from {supplier_b}) substitute for Ingredient A (from {s
```

```
Provide your structured JSON evaluation."""


def evaluate_substitutability(
    ingredient_a: str, supplier_a: str, compliance_a: dict,
    ingredient_b: str, supplier_b: str, compliance_b: dict,
    context_companies: list, context_bom_appearances: int,
    model_name: str = "gemini-flash-latest",
) -> dict:
    """
    Call Gemini to evaluate ingredient substitutability.
    Returns a parsed dict matching the JSON schema in AGNES_SYSTEM_PROMPT
    Falls back to a safe error dict if the response cannot be parsed.
    """
    user_message = _build_prompt(
        ingredient_a, supplier_a, compliance_a,
        ingredient_b, supplier_b, compliance_b,
        context_companies, context_bom_appearances,
    )

    # response_mime_type="application/json" tells Gemini to return
    # structured JSON directly — no markdown fences to strip.
    response = client.models.generate_content(
        model=model_name,
        contents=user_message,
        config=types.GenerateContentConfig(
            system_instruction=AGNES_SYSTEM_PROMPT,
            response_mime_type="application/json",
            temperature=0.2,   # low temp for consistent, factual reasoni
        ),
    )

    raw_text = response.text.strip()

    try:
        result = json.loads(raw_text)
    except json.JSONDecodeError as exc:
        result = {
            "substitutable":  False,
            "confidence":     0.0,
            "evidence_trail": [f"JSON parse error: {exc}", f"Raw: {raw_te
            "compliance_met": False,
            "compliance_gaps": ["LLM response could not be parsed — requi
            "reasoning":      "Parse failure.",
            "recommendation": "HUMAN_REVIEW_REQUIRED",
        }

    # Attach call metadata for traceability
    usage = response.usage_metadata
    result["_meta"] = {
        "model":          model_name,
        "input_tokens":   usage.prompt_token_count,
        "output_tokens":  usage.candidates_token_count,
        "ingredient_a":   ingredient_a,
        "supplier_a":     supplier_a,
        "ingredient_b":   ingredient_b,
        "supplier_b":     supplier_b,
    }
    return result
```

```
# ── Run Evaluation 1: Supplier consolidation (same ingredient) ──────────
companies_vd3      = df_target["company_name"].unique().tolist()
bom_total_vd3      = int(df_target["bom_appearances"].sum())
comp_prinova_chol  = scrape_supplier_compliance("Prinova USA", TARGET_ING
comp_purebulk_chol = scrape_supplier_compliance("PureBulk",   TARGET_INGR

print("Running Evaluation 1 — Supplier consolidation (same ingredient) …"
eval_within_cluster = evaluate_substitutability(
    ingredient_a=TARGET_INGREDIENT,  supplier_a="Prinova USA",  complianc
    ingredient_b=TARGET_INGREDIENT,  supplier_b="PureBulk",     complianc
    context_companies=companies_vd3, context_bom_appearances=bom_total_vd
)

print(f"\n=== Eval 1: Can PureBulk replace Prinova USA for {TARGET_INGRED
print(f"  Recommendation : {eval_within_cluster['recommendation']}")
print(f"  Substitutable  : {eval_within_cluster['substitutable']}")
print(f"  Confidence     : {eval_within_cluster['confidence']:.0%}")
print(f"  Compliance met : {eval_within_cluster['compliance_met']}")
if eval_within_cluster.get("compliance_gaps"):
    for gap in eval_within_cluster["compliance_gaps"]:
        print(f"  ⚠ Gap         : {gap}")
print("  Evidence trail:")
for fact in eval_within_cluster["evidence_trail"]:
    print(f"    • {fact}")
print(f"  Reasoning      : {eval_within_cluster['reasoning']}")
print(f"  Tokens used    : {eval_within_cluster['_meta']['input_tokens']}
      f"{eval_within_cluster['_meta']['output_tokens']} out")


# ── Run Evaluation 2: Cross-cluster (food-grade vs pharma-grade) ────────
companies_related  = df_related["company_name"].unique().tolist()
bom_total_related  = int(df_related["bom_appearances"].sum())
comp_prinova_vd3   = scrape_supplier_compliance("Prinova USA", RELATED_IN
comp_prinova_chol2 = scrape_supplier_compliance("Prinova USA", TARGET_ING

print("\nRunning Evaluation 2 — Cross-cluster grade comparison …")
eval_cross_cluster = evaluate_substitutability(
    ingredient_a=RELATED_INGREDIENT, supplier_a="Prinova USA", compliance
    ingredient_b=TARGET_INGREDIENT,  supplier_b="Prinova USA", compliance
    context_companies=companies_related, context_bom_appearances=bom_tota
)

print(f"\n=== Eval 2: Can pharma-grade cholecalciferol replace food-grade
print(f"  Recommendation : {eval_cross_cluster['recommendation']}")
print(f"  Substitutable  : {eval_cross_cluster['substitutable']}")
print(f"  Confidence     : {eval_cross_cluster['confidence']:.0%}")
print(f"  Compliance met : {eval_cross_cluster['compliance_met']}")
print(f"  Reasoning      : {eval_cross_cluster['reasoning']}")
```

```
Running Evaluation 1 — Supplier consolidation (same ingredient) …
```

```
=== Eval 1: Can PureBulk replace Prinova USA for vitamin-d3-cholecalcifero
l? ===
  Recommendation : REJECT
  Substitutable  : False
  Confidence     : 95%
  Compliance met : False
  ⚠ Gap         : Missing USP (United States Pharmacopeia) certification
  ⚠ Gap         : Missing Halal certification
  Evidence trail:
    • Ingredient A carries USP certification, which is a critical quality
standard for pharmaceutical-grade dietary supplements in the US market.
    • Ingredient A is Halal certified, whereas Ingredient B is not.
    • Ingredient B lacks third-party USP verification, which is often a no
n-negotiable requirement for major retailers like Kirkland Signature and G
NC.
    • Consolidation across 33 BOMs requires the replacement to meet the 'h
ighest common denominator' of certifications to avoid product-specific com
pliance failures.
  Reasoning      : While Ingredient B is listed as pharmaceutical grade, i
t fails to meet the compliance profile of Ingredient A due to the absence
of USP and Halal certifications. For major CPG brands like GNC and Kirklan
d, USP verification is a primary quality benchmark, and removing Halal sta
tus would invalidate the certification of any finished products currently
making that claim.
  Tokens used    : 783 in / 267 out

Running Evaluation 2 — Cross-cluster grade comparison …
=== Eval 2: Can pharma-grade cholecalciferol replace food-grade vitamin-d
3? ===
  Recommendation : APPROVE
  Substitutable  : True
  Confidence     : 98%
  Compliance met : True
  Reasoning      : Ingredient B is a direct quality upgrade from Ingredien
t A, moving from a food-grade blend to a pharmaceutical-grade USP cholecal
ciferol. Since Ingredient B meets or exceeds all regulatory, certification
, and potency requirements of Ingredient A while originating from the same
supplier, it is a low-risk candidate for consolidation.
```

In [7]:

```
# ────────────────────────────────────────────────────────────────
# CELL 6 — Optimization & Supplier Consolidation Algorithm
# ────────────────────────────────────────────────────────────────
# Ranking formula: consolidated_score = bom_appearances_covered × complia
#
# Approved recommendations (in order of confidence):
#   APPROVE                  → included, no conditions
#   APPROVE_WITH_CONDITIONS  → included, flagged
#   HUMAN_REVIEW_REQUIRED    → included as conditional, flagged
#   REJECT                   → excluded entirely
# ────────────────────────────────────────────────────────────────

_APPROVED_STATUSES = {"APPROVE", "APPROVE_WITH_CONDITIONS", "HUMAN_REVIEW


def compute_compliance_weight(compliance: dict) -> float:
    """Score a supplier's compliance profile. Higher = better. Range ~0.1
    weight = 1.0
    if compliance.get("grade") == "pharmaceutical":
        weight += 0.2
    elif compliance.get("grade") == "technical":
```

```
        weight -= 0.3
    if compliance.get("fda_registered"):
        weight += 0.1
    if compliance.get("non_gmo"):
        weight += 0.1
    cert_bonus = min(0.30, len(compliance.get("certifications", [])) * 0.
    weight += cert_bonus
    return round(max(0.1, weight), 3)


def consolidate_sourcing(
    ingredient_name: str,
    df_supplier_coverage: pd.DataFrame,
    approved_evaluations: list,
    scrape_fn,
) -> pd.DataFrame:
    """
    Produce a ranked sourcing recommendation for a given ingredient.
    Includes APPROVE, APPROVE_WITH_CONDITIONS, and HUMAN_REVIEW_REQUIRED.
    Excludes only REJECT.
    """
    # Debug: show what recommendations came back
    for ev in approved_evaluations:
        rec = ev.get("recommendation", "UNKNOWN")
        sup = ev["_meta"]["supplier_b"]
        print(f"  Evaluation result — {sup}: {rec}")

    approved_suppliers = {
        ev["_meta"]["supplier_b"]
        for ev in approved_evaluations
        if ev.get("recommendation") in _APPROVED_STATUSES
    }

    if not approved_suppliers:
        print(f"\nNo approvable suppliers for '{ingredient_name}' (all we
        return pd.DataFrame()

    df_ing = df_supplier_coverage[
        (df_supplier_coverage["ingredient_name"] == ingredient_name) &
        (df_supplier_coverage["supplier_name"].isin(approved_suppliers))
    ].copy()

    if df_ing.empty:
        print(f"No coverage data found for approved suppliers of '{ingred
        return pd.DataFrame()

    df_agg = (
        df_ing.groupby("supplier_name")
        .agg(
            bom_appearances_covered = ("bom_appearances", "sum"),
            companies_covered       = ("company_name",   "nunique"),
        )
        .reset_index()
    )

    # Build recommendation label per supplier
    rec_map = {
        ev["_meta"]["supplier_b"]: ev.get("recommendation", "UNKNOWN")
        for ev in approved_evaluations
    }
```

```
    rows = []
    for _, row in df_agg.iterrows():
        comp   = scrape_fn(row["supplier_name"], ingredient_name)
        weight = compute_compliance_weight(comp)
        score  = round(row["bom_appearances_covered"] * weight, 2)
        rows.append({
            "supplier_name":            row["supplier_name"],
            "bom_appearances_covered": int(row["bom_appearances_covered"]
            "companies_covered":        int(row["companies_covered"]),
            "grade":                    comp["grade"],
            "certifications":           ", ".join(comp["certifications"]),
            "lead_time_days":           comp["lead_time_days"],
            "fda_registered":           comp["fda_registered"],
            "non_gmo":                  comp["non_gmo"],
            "compliance_weight":        weight,
            "consolidated_score":       score,
            "llm_verdict":              rec_map.get(row["supplier_name"],
        })

    df_result = (
        pd.DataFrame(rows)
        .sort_values("consolidated_score", ascending=False)
        .reset_index(drop=True)
    )
    df_result.insert(0, "rank", df_result.index + 1)
    df_result["agnes_recommendation"] = df_result["rank"].map(
        {1: "PRIMARY SUPPLIER", 2: "SECONDARY / BACKUP"}
    ).fillna("MONITOR")

    return df_result


# ── Run consolidation ──────────────────────────────────────────────
print("Consolidation input — LLM verdicts:")
df_recommendation = consolidate_sourcing(
    ingredient_name=TARGET_INGREDIENT,
    df_supplier_coverage=df_supplier_coverage,
    approved_evaluations=[eval_within_cluster],
    scrape_fn=scrape_supplier_compliance,
)

print("\nSupplier ranking:")
display(df_recommendation)
```

```
Consolidation input — LLM verdicts:
  Evaluation result — PureBulk: REJECT

No approvable suppliers for 'vitamin-d3-cholecalciferol' (all were REJEC
T).

Supplier ranking:
```

In [8]:

```
# ────────────────────────────────────────────────────────────────
# CELL 7 — Final Sourcing Recommendation Output
# ────────────────────────────────────────────────────────────────

SEP  = "=" * 70
SEP2 = "-" * 70
```

```
print(SEP)
print("  AGNES — CONSOLIDATED SOURCING RECOMMENDATION")
print(SEP)
print(f"  Ingredient    : {TARGET_INGREDIENT}")
print(f"  Scope         : {df_target['company_name'].nunique()} CPG compan
      f"{int(df_target['bom_appearances'].sum())} BOM appearances")
print(f"  LLM model     : {eval_within_cluster['_meta']['model']}")
print(SEP)

# ── Recommendation table ──────────────────────────────────────────
if df_recommendation.empty:
    print("\nNo suppliers passed the consolidation filter.")
else:
    print("\nSupplier Ranking:")
    display(
        df_recommendation[[
            "rank", "supplier_name", "bom_appearances_covered", "companie
            "grade", "certifications", "lead_time_days",
            "compliance_weight", "consolidated_score", "llm_verdict", "ag
        ]]
    )

    winner = df_recommendation.iloc[0]
    print(f"\n{SEP2}")
    print("RECOMMENDED ACTION")
    print(SEP2)
    print(f"  Consolidate all {df_target['company_sku'].nunique()} compan
    print(f"  into a single purchase agreement with: {winner['supplier_na
    print(f"  Volume coverage  : {winner['companies_covered']} companies,
          f"{winner['bom_appearances_covered']} BOM appearances")
    print(f"  Grade            : {winner['grade']}")
    print(f"  Certifications   : {winner['certifications']}")
    print(f"  Lead time        : {winner['lead_time_days']} days")
    print(f"  Compliance score : {winner['compliance_weight']}")
    print(f"  Agnes score      : {winner['consolidated_score']}")
    print(f"  LLM verdict      : {winner['llm_verdict']}")

# ── Evidence trail ────────────────────────────────────────────────
print(f"\n{SEP2}")
print("EVIDENCE TRAIL (AI Reasoning — Evaluation 1: Supplier Consolidatio
print(SEP2)
for i, fact in enumerate(eval_within_cluster["evidence_trail"], 1):
    print(f"  [{i}] {fact}")
print(f"\n  LLM confidence   : {eval_within_cluster['confidence']:.0%}")
print(f"  Compliance status: {'PASSED' if eval_within_cluster['compliance
if eval_within_cluster.get("compliance_gaps"):
    for gap in eval_within_cluster["compliance_gaps"]:
        print(f"  ⚠ Gap: {gap}")
print(f"\n  Reasoning summary:")
print(f"  {eval_within_cluster['reasoning']}")

# ── Cross-cluster evaluation summary ──────────────────────────────
print(f"\n{SEP2}")
print("EVIDENCE TRAIL (AI Reasoning — Evaluation 2: Cross-Cluster Grade C
print(SEP2)
for i, fact in enumerate(eval_cross_cluster["evidence_trail"], 1):
    print(f"  [{i}] {fact}")
print(f"\n  Recommendation   : {eval_cross_cluster['recommendation']}")
print(f"  Reasoning summary:")
```

```
print(f"  {eval_cross_cluster['reasoning']}")

# ── Fragmentation waste summary ──────────────────────────────────────
print(f"\n{SEP2}")
print("FRAGMENTATION ANALYSIS")
print(SEP2)
print(f"  {df_target['company_name'].nunique()} CPG companies buying the
print(f"  across {df_target['company_sku'].nunique()} unique SKUs")
print(f"  from only {df_target_suppliers['supplier_name'].nunique()} supp
print(f"  → Zero combined-volume leverage today")
if not df_recommendation.empty:
    winner = df_recommendation.iloc[0]
    print(f"  → Agnes recommendation: single consolidated contract with {
    print(f"    Est. combined demand : {winner['bom_appearances_covered']
print(SEP)
```

```
======================================================================
  AGNES — CONSOLIDATED SOURCING RECOMMENDATION
======================================================================
  Ingredient   : vitamin-d3-cholecalciferol
  Scope        : 17 CPG companies, 33 BOM appearances
  LLM model    : gemini-flash-latest
======================================================================

No suppliers passed the consolidation filter.

──────────────────────────────────────────────────────────────────────
EVIDENCE TRAIL (AI Reasoning — Evaluation 1: Supplier Consolidation)
──────────────────────────────────────────────────────────────────────
  [1] Ingredient A carries USP certification, which is a critical quality
standard for pharmaceutical-grade dietary supplements in the US market.
  [2] Ingredient A is Halal certified, whereas Ingredient B is not.
  [3] Ingredient B lacks third-party USP verification, which is often a no
n-negotiable requirement for major retailers like Kirkland Signature and G
NC.
  [4] Consolidation across 33 BOMs requires the replacement to meet the 'h
ighest common denominator' of certifications to avoid product-specific com
pliance failures.

  LLM confidence   : 95%
  Compliance status: GAPS IDENTIFIED
  ⚠ Gap: Missing USP (United States Pharmacopeia) certification
  ⚠ Gap: Missing Halal certification

  Reasoning summary:
  While Ingredient B is listed as pharmaceutical grade, it fails to meet t
he compliance profile of Ingredient A due to the absence of USP and Halal
certifications. For major CPG brands like GNC and Kirkland, USP verificati
on is a primary quality benchmark, and removing Halal status would invalid
ate the certification of any finished products currently making that claim
.

──────────────────────────────────────────────────────────────────────
EVIDENCE TRAIL (AI Reasoning — Evaluation 2: Cross-Cluster Grade Check)
──────────────────────────────────────────────────────────────────────
  [1] Both ingredients are Vitamin D3 (Cholecalciferol) sourced from the s
ame supplier (Prinova USA).
  [2] Ingredient B is Pharmaceutical grade (USP), which exceeds the Food g
rade specification of Ingredient A.
  [3] Ingredient B maintains all certifications held by Ingredient A, incl
uding GMP, Kosher, and Non-GMO status.
  [4] Ingredient B adds Halal certification and USP monograph compliance,
which were absent in Ingredient A.
  [5] Lead times are identical at 14 days, posing no risk to supply chain
continuity.
  [6] The affected brands (e.g., Nature Made, Kirkland Signature) typicall
y require USP-grade ingredients for dietary supplement labeling, which Ing
redient B provides.

  Recommendation   : APPROVE
  Reasoning summary:
  Ingredient B is a direct quality upgrade from Ingredient A, moving from
a food-grade blend to a pharmaceutical-grade USP cholecalciferol. Since In
gredient B meets or exceeds all regulatory, certification, and potency req
uirements of Ingredient A while originating from the same supplier, it is
a low-risk candidate for consolidation.
```

```
──────────────────────────────────────────────────────────────────────
FRAGMENTATION ANALYSIS
──────────────────────────────────────────────────────────────────────
  17 CPG companies buying the SAME ingredient separately
  across 17 unique SKUs
  from only 2 suppliers
  → Zero combined-volume leverage today
======================================================================
```

In [9]:

```python
# ────────────────────────────────────────────────────────────────────
# CELL 8 — Go-Fish Supplier Trust Score
# ────────────────────────────────────────────────────────────────────
# Tracks supplier reliability like a go-fish card game:
#   • More on-time deliveries → score increases (+10 per delivery)
#   • Delays / quality incidents → score decreases (−20 per event)
#
# No historical order data exists in the DB, so we generate
# deterministic mock delivery history seeded by supplier name.
# In production Agnes would pull from an ERP / TMS system.
# ────────────────────────────────────────────────────────────────────

class SupplierTrustTracker:
    BASE_SCORE    = 100
    ON_TIME_BONUS = 10
    DELAY_PENALTY = 20

    def __init__(self):
        self._scores  = {}
        self._history = {}

    def simulate_history(self, supplier_name: str, n_deliveries: int = 20
        rng   = random.Random(hash(supplier_name) % (2 ** 31))
        score = self.BASE_SCORE
        events = []
        for _ in range(n_deliveries):
            on_time = rng.random() < 0.80   # 80 % base on-time rate
            if on_time:
                score += self.ON_TIME_BONUS
                events.append("on_time")
            else:
                score -= self.DELAY_PENALTY
                events.append("delay")
        self._scores[supplier_name]  = max(10, score)
        self._history[supplier_name] = events

    def get_score(self, supplier_name: str) -> int:
        if supplier_name not in self._scores:
            self.simulate_history(supplier_name)
        return self._scores[supplier_name]

    def get_trust_multiplier(self, supplier_name: str) -> float:
        return round(max(0.5, min(1.5, self.get_score(supplier_name) / 10

    def get_trust_tier(self, supplier_name: str) -> str:
        s = self.get_score(supplier_name)
        if s >= 160: return "PLATINUM"
        if s >= 130: return "GOLD"
        if s >= 100: return "SILVER"
        if s >= 70:  return "BRONZE"
```

```
        return "PROBATION"


# ── Initialise & simulate for all 40 suppliers ──────────────────────────
trust_tracker = SupplierTrustTracker()

conn = sqlite3.connect(DB_PATH)
all_suppliers = pd.read_sql_query("SELECT Id, Name FROM Supplier ORDER BY
conn.close()

for name in all_suppliers["Name"]:
    trust_tracker.simulate_history(name, n_deliveries=20)

# ── Build leaderboard ───────────────────────────────────────────────────
trust_rows = []
for name in all_suppliers["Name"]:
    score   = trust_tracker.get_score(name)
    history = trust_tracker._history[name]
    on_time = history.count("on_time")
    delays  = history.count("delay")
    trust_rows.append({
        "supplier_name":      name,
        "trust_score":        score,
        "trust_tier":         trust_tracker.get_trust_tier(name),
        "trust_multiplier":   trust_tracker.get_trust_multiplier(name),
        "on_time_deliveries": on_time,
        "delays":             delays,
        "on_time_rate":       f"{on_time / 20:.0%}",
    })

df_trust = (
    pd.DataFrame(trust_rows)
    .sort_values("trust_score", ascending=False)
    .reset_index(drop=True)
)
df_trust.insert(0, "rank", df_trust.index + 1)

SEP = "=" * 70
print(SEP)
print("  AGNES — SUPPLIER TRUST LEADERBOARD  (Go-Fish Reliability Score)"
print(SEP)
print(f"  Base score     : {SupplierTrustTracker.BASE_SCORE}")
print(f"  On-time bonus  : +{SupplierTrustTracker.ON_TIME_BONUS} per deli
print(f"  Delay penalty  : -{SupplierTrustTracker.DELAY_PENALTY} per inci
print(f"  Tiers: PROBATION (<70) → BRONZE → SILVER → GOLD → PLATINUM (≥16
print(SEP)
display(df_trust)

# ── Show trust impact on vitamin-d3-cholecalciferol suppliers ──────────
print(f"\nTrust-adjusted compliance weights for '{TARGET_INGREDIENT}' sup
for s in ["Prinova USA", "PureBulk"]:
    score  = trust_tracker.get_score(s)
    mult   = trust_tracker.get_trust_multiplier(s)
    tier   = trust_tracker.get_trust_tier(s)
    comp   = scrape_supplier_compliance(s, TARGET_INGREDIENT)
    base_w = compute_compliance_weight(comp)
    adj_w  = round(base_w * mult, 3)
    print(f"  {s:15s} | score={score:4d} | tier={tier:9s} | "
          f"compliance={base_w:.3f} × trust={mult:.3f} → adjusted={adj_w:
```

```
======================================================================
  AGNES — SUPPLIER TRUST LEADERBOARD  (Go-Fish Reliability Score)
======================================================================
  Base score     : 100
  On-time bonus  : +10 per delivery
  Delay penalty  : −20 per incident
  Tiers: PROBATION (<70) → BRONZE → SILVER → GOLD → PLATINUM (≥160)
======================================================================
```

	rank	supplier_name	trust_score	trust_tier	trust_multiplier	on_time_deliveries
0	1	Stauber	300	PLATINUM	1.5	20
1	2	AIDP	270	PLATINUM	1.5	19
2	3	Virginia Dare	270	PLATINUM	1.5	19
3	4	Source-Omega LLC	270	PLATINUM	1.5	19
4	5	Trace Minerals	270	PLATINUM	1.5	19
5	6	Actus Nutrition	240	PLATINUM	1.5	18
6	7	American Botanicals	240	PLATINUM	1.5	18
7	8	BulkSupplements	240	PLATINUM	1.5	18
8	9	Sawgrass Nutra Labs	240	PLATINUM	1.5	18
9	10	Custom Probiotics	240	PLATINUM	1.5	18
10	11	Nutri Avenue	240	PLATINUM	1.5	18
11	12	Vitaquest	240	PLATINUM	1.5	18
12	13	Cargill	240	PLATINUM	1.5	18
13	14	Sensient	210	PLATINUM	1.5	17
14	15	Ashland	210	PLATINUM	1.5	17
15	16	Cambrex	210	PLATINUM	1.5	17
16	17	Koster Keunen	210	PLATINUM	1.5	17
17	18	Mueggenburg USA	210	PLATINUM	1.5	17
18	19	Gold Coast Ingredients	210	PLATINUM	1.5	17
19	20	Icelandirect	210	PLATINUM	1.5	17
20	21	Univar Solutions	210	PLATINUM	1.5	17
21	22	Strahl & Pitsch	210	PLATINUM	1.5	17
22	23	Balchem	180	PLATINUM	1.5	16
23	24	ADM	180	PLATINUM	1.5	16
24	25	Capsuline	180	PLATINUM	1.5	16
25	26	IFF	180	PLATINUM	1.5	16
26	27	PureBulk	180	PLATINUM	1.5	16
27	28	Nutra Food Ingredients	180	PLATINUM	1.5	16
28	29	TCI America	180	PLATINUM	1.5	16
29	30	Specialty Enzymes & Probiotics	180	PLATINUM	1.5	16

	rank	supplier_name	trust_score	trust_tier	trust_multiplier	on_time_deliveries
30	31	Spectrum Chemical	180	PLATINUM	1.5	16
31	32	Darling Ingredients / Rousselot	180	PLATINUM	1.5	16
32	33	Ingredion	150	GOLD	1.5	15
33	34	FutureCeuticals	150	GOLD	1.5	15
34	35	Makers Nutrition	150	GOLD	1.5	15
35	36	Jost Chemical	150	GOLD	1.5	15
36	37	Colorcon	150	GOLD	1.5	15
37	38	Nutra Blend	120	SILVER	1.2	14
38	39	Prinova USA	120	SILVER	1.2	14
39	40	Magtein / ThreoTech LLC	90	BRONZE	0.9	13

```
Trust-adjusted compliance weights for 'vitamin-d3-cholecalciferol' supplie
rs:
  Prinova USA      | score= 120 | tier=SILVER    | compliance=1.600 × trust
=1.200 → adjusted=1.920
  PureBulk         | score= 180 | tier=PLATINUM  | compliance=1.500 × trust
=1.500 → adjusted=2.250
```

In [10]:

```python
# ────────────────────────────────────────────────────────────────
# CELL 9 — Supply Chain Risk Heat Map (Vulnerability Index)
# ────────────────────────────────────────────────────────────────
# Scans all 143 fragmented ingredients and scores each by:
#
#   vulnerability_index = total_bom_appearances / distinct_supplier_count
#
# Higher index = more dangerous: high demand concentrated in few supplier
# Single-source ingredients are flagged CRITICAL regardless of volume.
# ────────────────────────────────────────────────────────────────

# ── Supplier count per ingredient (from coverage data already loaded) ──
df_supplier_counts = (
    df_supplier_coverage
    .groupby("ingredient_name")["supplier_name"]
    .nunique()
    .reset_index()
    .rename(columns={"supplier_name": "supplier_count"})
)

# ── BOM totals per ingredient ──────────────────────────────────────
df_bom_totals = (
    df_fragmented
    .groupby("ingredient_name")
    .agg(
        total_bom_appearances = ("bom_appearances", "sum"),
        company_count         = ("company_name",    "nunique"),
    )
    .reset_index()
)
```

```
# ── Merge and score ──────────────────────────────────────────────────────
df_risk = df_bom_totals.merge(df_supplier_counts, on="ingredient_name")
df_risk["vulnerability_index"] = (
    df_risk["total_bom_appearances"] / df_risk["supplier_count"]
).round(2)


def _risk_tier(row: pd.Series) -> str:
    if row["supplier_count"] == 1:
        return "CRITICAL"
    if row["supplier_count"] == 2 and row["total_bom_appearances"] >= 20:
        return "HIGH"
    if row["total_bom_appearances"] >= 15:
        return "MEDIUM"
    return "LOW"


df_risk["risk_tier"] = df_risk.apply(_risk_tier, axis=1)
df_risk = df_risk.sort_values("vulnerability_index", ascending=False).res
df_risk.insert(0, "rank", df_risk.index + 1)

# ── Summary ──────────────────────────────────────────────────────────────
SEP = "=" * 70
print(SEP)
print("  AGNES — SUPPLY CHAIN RISK HEAT MAP")
print(SEP)
print(f"  Total ingredients analyzed : {len(df_risk)}")
print(f"  CRITICAL  (1 supplier)     : {(df_risk['risk_tier'] == 'CRITICA
print(f"  HIGH      (2 sup, ≥20 BOM) : {(df_risk['risk_tier'] == 'HIGH').
print(f"  MEDIUM    (≥15 BOMs)       : {(df_risk['risk_tier'] == 'MEDIUM'
print(f"  LOW                        : {(df_risk['risk_tier'] == 'LOW').s
print(SEP)
print("\nTop 15 most vulnerable ingredients:")
display(
    df_risk.head(15)[[
        "rank", "ingredient_name", "total_bom_appearances",
        "company_count", "supplier_count", "vulnerability_index", "risk_t
    ]]
)
```

```
======================================================================
  AGNES — SUPPLY CHAIN RISK HEAT MAP
======================================================================
  Total ingredients analyzed : 143
  CRITICAL  (1 supplier)     : 18
  HIGH      (2 sup, ≥20 BOM) : 11
  MEDIUM    (≥15 BOMs)       : 9
  LOW                        : 105
======================================================================

Top 15 most vulnerable ingredients:
```

	rank	ingredient_name	total_bom_appearances	company_count	supplier_count	vuln
0	1	maltodextrin	21	8	1	
1	2	glycerin	17	8	1	
2	3	vitamin-d3-cholecalciferol	33	17	2	
3	4	magnesium-stearate	30	11	2	
4	5	gelatin	30	11	2	
5	6	microcrystalline-cellulose	29	13	2	
6	7	citric-acid	26	12	2	
7	8	natural-flavor	13	8	1	
8	9	silicon-dioxide	25	10	2	
9	10	magnesium-oxide	25	10	2	
10	11	vitamin-c	24	13	2	
11	12	stearic-acid	24	7	2	
12	13	calcium-carbonate	20	8	2	
13	14	whey-protein-isolate	20	11	2	
14	15	croscarmellose-sodium	19	8	2	

In [11]:
```
# ────────────────────────────────────────────────────────────────
# CELL 10 — Supply Chain Disruption Simulator
# ────────────────────────────────────────────────────────────────
# "What if Prinova USA goes offline tomorrow?"
# Agnes identifies all affected ingredients, companies, and BOMs,
# finds alternate suppliers, and calls Gemini once for a consolidated
# contingency recommendation — not once per ingredient.
# ────────────────────────────────────────────────────────────────

DISRUPTED_SUPPLIER = "Prinova USA"

# ── Query 1: All ingredients covered by the disrupted supplier ──────
SQL_DISRUPTED = """
WITH parsed AS (
    SELECT
        p.Id AS product_id,
        p.CompanyId,
        c.Name AS company_name,
        SUBSTR(
            SUBSTR(p.SKU, 4 + INSTR(SUBSTR(p.SKU, 4), '-')),
            1,
            LENGTH(SUBSTR(p.SKU, 4 + INSTR(SUBSTR(p.SKU, 4), '-'))) - 9
        ) AS ingredient_name
    FROM Product p
    JOIN Company c ON c.Id = p.CompanyId
    WHERE p.Type = 'raw-material'
),
```

```
bom_usage AS (
    SELECT
        pr.ingredient_name,
        pr.company_name,
        pr.product_id,
        COUNT(bc.BOMId) AS bom_appearances
    FROM parsed pr
    JOIN BOM_Component bc ON bc.ConsumedProductId = pr.product_id
    GROUP BY pr.product_id
)
SELECT
    bu.ingredient_name,
    SUM(bu.bom_appearances)      AS bom_appearances_at_risk,
    COUNT(DISTINCT bu.company_name) AS companies_at_risk
FROM bom_usage bu
JOIN Supplier_Product sp ON sp.ProductId = bu.product_id
JOIN Supplier s           ON s.Id        = sp.SupplierId
WHERE s.Name = :supplier
GROUP BY bu.ingredient_name
ORDER BY bom_appearances_at_risk DESC
"""

# ── Query 2: Alternate suppliers per ingredient (excluding disrupted one
SQL_ALTERNATES = """
WITH parsed AS (
    SELECT
        p.Id AS product_id,
        SUBSTR(
            SUBSTR(p.SKU, 4 + INSTR(SUBSTR(p.SKU, 4), '-')),
            1,
            LENGTH(SUBSTR(p.SKU, 4 + INSTR(SUBSTR(p.SKU, 4), '-'))) - 9
        ) AS ingredient_name
    FROM Product p
    WHERE p.Type = 'raw-material'
)
SELECT DISTINCT
    pr.ingredient_name,
    s.Name AS alternate_supplier
FROM parsed pr
JOIN Supplier_Product sp ON sp.ProductId = pr.product_id
JOIN Supplier s           ON s.Id        = sp.SupplierId
WHERE s.Name != :supplier
"""

conn = sqlite3.connect(DB_PATH)
df_disrupted  = pd.read_sql_query(SQL_DISRUPTED,  conn, params={"supplier
df_alternates = pd.read_sql_query(SQL_ALTERNATES, conn, params={"supplier
conn.close()

# ── Annotate with backup availability ──────────────────────────────────
alt_map = df_alternates.groupby("ingredient_name")["alternate_supplier"].

df_disrupted["alternate_suppliers"] = df_disrupted["ingredient_name"].map
    lambda ing: ", ".join(alt_map.get(ing, [])) or "NONE"
)
df_disrupted["has_backup"] = df_disrupted["ingredient_name"].map(
    lambda ing: len(alt_map.get(ing, [])) > 0
)
df_disrupted["exposure"] = df_disrupted["has_backup"].map(
    {True: "MANAGEABLE", False: "CRITICAL EXPOSURE"}
```

```
)

critical_count = int((~df_disrupted["has_backup"]).sum())

SEP = "=" * 70
print(SEP)
print(f"  AGNES — DISRUPTION SIMULATOR: '{DISRUPTED_SUPPLIER}' goes offli
print(SEP)
print(f"  Ingredients directly affected : {len(df_disrupted)}")
print(f"  Total BOM appearances at risk : {int(df_disrupted['bom_appearan
print(f"  Ingredients with NO backup    : {critical_count}  ← CRITICAL EX
print(SEP)
display(
    df_disrupted[[
        "ingredient_name", "bom_appearances_at_risk",
        "companies_at_risk", "alternate_suppliers", "exposure"
    ]].head(20)
)

# ── Single Gemini call: consolidated contingency plan ──────────────────
top_critical   = df_disrupted[~df_disrupted["has_backup"]].head(3)
top_manageable = df_disrupted[df_disrupted["has_backup"]].head(3)

critical_list = "\n".join(
    f"  - {r['ingredient_name']} ({r['bom_appearances_at_risk']} BOM appe
    for _, r in top_critical.iterrows()
) or "  None"

manageable_list = "\n".join(
    f"  - {r['ingredient_name']} ({r['bom_appearances_at_risk']} BOMs → b
    for _, r in top_manageable.iterrows()
)

disruption_prompt = f"""## Supply Chain Disruption Alert

{DISRUPTED_SUPPLIER} has gone offline due to a facility shutdown.

### Impact Summary
- {len(df_disrupted)} ingredients affected
- {int(df_disrupted['bom_appearances_at_risk'].sum())} total BOM appearan
- {critical_count} ingredients have NO alternate supplier (critical expos

### Critical Exposure — no backup available:
{critical_list}

### Manageable — backup suppliers exist:
{manageable_list}

### Task
Provide an immediate contingency plan for CPG procurement teams.

Respond in JSON:
{{
  "risk_level": "<CRITICAL | HIGH | MEDIUM>",
  "highest_priority_ingredient": "<name>",
  "immediate_actions": ["<action 1>", "<action 2>", "<action 3>"],
  "week_1_actions": ["<action 1>", "<action 2>"],
  "month_1_actions": ["<action 1>", "<action 2>"],
  "strategic_recommendation": "<2-3 sentences on long-term sourcing strat
}}"""
```

```
print(f"\nCalling Gemini for contingency plan …")
disruption_resp = client.models.generate_content(
    model="gemini-flash-latest",
    contents=disruption_prompt,
    config=types.GenerateContentConfig(
        response_mime_type="application/json",
        temperature=0.2,
    ),
)
contingency = json.loads(disruption_resp.text.strip())

SEP2 = "—" * 70
print(f"\n{SEP2}")
print("AGNES CONTINGENCY PLAN")
print(SEP2)
print(f"  Risk level         : {contingency.get('risk_level', 'N/A')}")
print(f"  Highest priority   : {contingency.get('highest_priority_ingredi
print(f"\n  IMMEDIATE (24 h):")
for a in contingency.get("immediate_actions", []):
    print(f"    • {a}")
print(f"\n  WEEK 1:")
for a in contingency.get("week_1_actions", []):
    print(f"    • {a}")
print(f"\n  MONTH 1:")
for a in contingency.get("month_1_actions", []):
    print(f"    • {a}")
print(f"\n  Strategic recommendation:")
print(f"  {contingency.get('strategic_recommendation', '')}")
```

```
======================================================================
  AGNES — DISRUPTION SIMULATOR: 'Prinova USA' goes offline
======================================================================
  Ingredients directly affected : 135
  Total BOM appearances at risk : 712
  Ingredients with NO backup    : 0  ← CRITICAL EXPOSURE
======================================================================
```

	ingredient_name	bom_appearances_at_risk	companies_at_risk	alternate_suppliers	
0	vitamin-d3-cholecalciferol	33	17	PureBulk	M
1	citric-acid	26	12	Univar Solutions	M
2	magnesium-oxide	25	10	Jost Chemical	M
3	vitamin-c	24	13	PureBulk	M
4	whey-protein-isolate	20	11	Actus Nutrition	M
5	vitamin-e	19	10	PureBulk	M
6	vitamin-a	19	10	PureBulk	M
7	riboflavin	18	8	PureBulk	M
8	biotin	18	6	PureBulk	M
9	zinc	17	11	Jost Chemical	M
10	whey-protein-concentrate	17	8	Actus Nutrition	M
11	acesulfame-potassium	17	8	Cargill	M
12	vitamin-d3	14	8	PureBulk	M
13	magnesium	14	11	Jost Chemical	M
14	vitamin-d	13	7	PureBulk	M
15	ascorbic-acid	13	7	PureBulk	M
16	sucralose	11	9	Univar Solutions	M
17	folic-acid	11	3	PureBulk	M
18	cyanocobalamin	11	5	PureBulk	M
19	b-vitamins	11	9	PureBulk	M

```
Calling Gemini for contingency plan …
```

```
────────────────────────────────────────────────────────────────────────────────
AGNES CONTINGENCY PLAN
────────────────────────────────────────────────────────────────────────────────
  Risk level         : HIGH
  Highest priority   : vitamin-d3-cholecalciferol

  IMMEDIATE (24 h):
    • Contact PureBulk, Univar Solutions, and Jost Chemical to secure exis
ting spot-buy inventory.
    • Place immediate bridge orders for the top 10 ingredients by BOM volu
me to prevent production line stoppages.
    • Issue a formal notice to the production planning team regarding pote
ntial lead-time extensions for 712 BOMs.

  WEEK 1:
    • Conduct a comprehensive audit of the remaining 132 ingredients to ve
rify and activate all secondary supplier contracts.
    • Validate the quality specifications of backup suppliers against exis
ting Prinova-sourced COAs to ensure formulation compatibility.

  MONTH 1:
    • Onboard and qualify at least one additional tertiary supplier for hi
gh-volume ingredients like Citric Acid and Vitamin D3.
    • Perform a post-mortem on safety stock levels and adjust minimum inve
ntory thresholds for all 135 affected ingredients.

  Strategic recommendation:
  Shift from a single-distributor reliance to a diversified multi-sourcing
strategy that distributes volume across at least two geographically distin
ct suppliers per ingredient. Implement a robust supplier risk monitoring s
ystem to provide early warnings on facility-specific shutdowns and logisti
cs bottlenecks.
```

In [12]:

```
# ────────────────────────────────────────────────────────────────
# CELL 11 — Buying Consortium Recommender (GPO Opportunities)
# ────────────────────────────────────────────────────────────────
# Identifies clusters of CPG companies buying the same ingredient
# independently from the same limited supplier pool.
# Recommends forming a Group Purchasing Organization (GPO) to
# unlock collective negotiating leverage.
#
# consortium_leverage_score = total_bom_appearances × company_count × 0.1
# (proxy for estimated negotiating leverage gain)
# ────────────────────────────────────────────────────────────────

# ── Build candidate ingredients: 5+ companies, narrow supplier pool ────
df_ingredient_summary = (
    df_fragmented
    .groupby("ingredient_name")
    .agg(
        company_count         = ("company_name", "nunique"),
        total_bom_appearances = ("bom_appearances", "sum"),
        companies             = ("company_name", lambda x: sorted(x.uniqu
    )
    .reset_index()
)

df_ingredient_summary = df_ingredient_summary.merge(df_supplier_counts, o

df_consortium = df_ingredient_summary[
```

```
    df_ingredient_summary["company_count"] >= 5
].copy()

df_consortium["consortium_leverage_score"] = (
    df_consortium["total_bom_appearances"] * df_consortium["company_count
).round(1)

df_consortium["est_leverage_gain"] = df_consortium["company_count"].map(
    lambda n: f"+{min(25, round(n * 1.5)):.0f}% negotiating leverage"
)

df_consortium = (
    df_consortium
    .sort_values("consortium_leverage_score", ascending=False)
    .reset_index(drop=True)
)
df_consortium.insert(0, "rank", df_consortium.index + 1)

SEP = "=" * 70
print(SEP)
print("  AGNES — BUYING CONSORTIUM RECOMMENDATIONS (GPO Opportunities)")
print(SEP)
print(f"  Consortium candidates : {len(df_consortium)}")
print(f"  (ingredients bought independently by 5+ CPG companies)")
print(SEP)
print("\nTop 10 Group Purchasing Opportunities:")
display(
    df_consortium.head(10)[[
        "rank", "ingredient_name", "company_count", "total_bom_appearance
        "supplier_count", "consortium_leverage_score", "est_leverage_gain
    ]]
)

# ── LLM call: GPO recommendation for the #1 opportunity ──────────────
top = df_consortium.iloc[0]
shown_companies = top["companies"][:8]
company_str = ", ".join(shown_companies)
if len(top["companies"]) > 8:
    company_str += f" … (+{len(top['companies']) - 8} more)"

consortium_prompt = f"""## Group Purchasing Organization Opportunity

Agnes has identified a major buying consortium opportunity.

### Ingredient: {top['ingredient_name']}
- Companies buying independently : {top['company_count']} ({company_str})
- Total BOM appearances          : {top['total_bom_appearances']}
- Available suppliers            : {top['supplier_count']} (limited pool)
- Status today                   : Each company negotiates alone → zero c

### Task
Agnes recommends these companies form a Group Purchasing Organization (GP
Provide the business case, structure, and first action step.

Respond in JSON:
{{
  "gpo_recommendation": "<2-3 sentence executive summary>",
  "estimated_savings_pct": "<X-Y% range>",
  "structure_suggestion": "<how to organize the GPO: lead buyer, shared c
  "key_risks": ["<risk 1>", "<risk 2>", "<risk 3>"],
```

```
  "first_step": "<the single most important first action>"
}}"""

print(f"\nCalling Gemini for GPO recommendation on '{top['ingredient_name
gpo_resp = client.models.generate_content(
    model="gemini-flash-latest",
    contents=consortium_prompt,
    config=types.GenerateContentConfig(
        response_mime_type="application/json",
        temperature=0.2,
    ),
)
gpo = json.loads(gpo_resp.text.strip())

SEP2 = "—" * 70
print(f"\n{SEP2}")
print(f"GPO RECOMMENDATION: {top['ingredient_name'].upper()}")
print(SEP2)
print(f"  Companies in consortium : {top['company_count']}")
print(f"  Combined BOM demand     : {top['total_bom_appearances']} appear
print(f"  Estimated savings       : {gpo.get('estimated_savings_pct', 'N/
print(f"\n  Recommendation:")
print(f"  {gpo.get('gpo_recommendation', '')}")
print(f"\n  Structure:")
print(f"  {gpo.get('structure_suggestion', '')}")
print(f"\n  Key risks:")
for r in gpo.get("key_risks", []):
    print(f"    • {r}")
print(f"\n  First step: {gpo.get('first_step', '')}")
```

```
======================================================================
  AGNES — BUYING CONSORTIUM RECOMMENDATIONS (GPO Opportunities)
======================================================================
  Consortium candidates : 51
  (ingredients bought independently by 5+ CPG companies)
======================================================================

Top 10 Group Purchasing Opportunities:
```

	rank	ingredient_name	company_count	total_bom_appearances	supplier_count	consc
0	1	vitamin-d3-cholecalciferol	17	33	2	
1	2	microcrystalline-cellulose	13	29	2	
2	3	gelatin	11	30	2	
3	4	magnesium-stearate	11	30	2	
4	5	vitamin-c	13	24	2	
5	6	citric-acid	12	26	2	
6	7	magnesium-oxide	10	25	2	
7	8	silicon-dioxide	10	25	2	
8	9	whey-protein-isolate	11	20	2	
9	10	vitamin-a	10	19	2	

```
Calling Gemini for GPO recommendation on 'vitamin-d3-cholecalciferol' …
────────────────────────────────────────────────────────────────────────
GPO RECOMMENDATION: VITAMIN-D3-CHOLECALCIFEROL
────────────────────────────────────────────────────────────────────────
  Companies in consortium : 17
  Combined BOM demand     : 33 appearances
  Estimated savings       : 12-22%

  Recommendation:
  Consolidating the purchasing power of 17 major supplement brands for Vit
amin D3 will shift the market dynamic from fragmented buyers to a unified
block. Given the limited supplier pool of only two providers, a GPO provid
es the necessary scale to secure volume discounts and ensure supply priori
ty in a constrained market.

  Structure:
  A Third-Party Managed GPO model where a neutral entity negotiates a Mast
er Service Agreement (MSA) with the two suppliers, allowing individual com
panies to issue purchase orders against the pre-negotiated volume-based pr
ice tiers.

  Key risks:
    • Anti-trust and price-fixing regulatory scrutiny
    • Supply chain vulnerability due to extreme concentration on only two
suppliers
    • Difficulty in aligning quality specifications and testing protocols
across 17 different brands

  First step: Execute a multi-party Non-Disclosure Agreement (NDA) to cond
uct a confidential spend analysis and aggregate the total annual tonnage r
equirements across all 17 participants.
```

In [13]:
```
# ────────────────────────────────────────────────────────────────────
# CELL 12 — Agnes 2.0 Executive Dashboard (All 143 Ingredients)
# ────────────────────────────────────────────────────────────────────
# Aggregates every Agnes signal into a single executive summary:
#   • Fragmentation demand  (Cell 2)
#   • Risk tier             (Cell 9)
#   • Best supplier trust   (Cell 8)
#   • Best compliance weight(Cell 4 + 6)
#   • GPO eligibility       (Cell 11)
#   • Estimated savings     (mocked: 8–15 % per consolidated ingredient)
#
# No LLM calls — pure heuristic scoring across all 143 ingredients
# for speed and cost efficiency during the demo.
# ────────────────────────────────────────────────────────────────────

def _best_trust_mult(ingredient_name: str) -> float:
    suppliers = df_supplier_coverage[
        df_supplier_coverage["ingredient_name"] == ingredient_name
    ]["supplier_name"].unique()
    if len(suppliers) == 0:
        return 1.0
    return max(trust_tracker.get_trust_multiplier(s) for s in suppliers)


def _best_compliance_weight(ingredient_name: str) -> float:
    suppliers = df_supplier_coverage[
        df_supplier_coverage["ingredient_name"] == ingredient_name
    ]["supplier_name"].unique()
    if len(suppliers) == 0:
        return 1.0
    return max(
        compute_compliance_weight(scrape_supplier_compliance(s, ingredien
        for s in suppliers
    )


consortium_set = set(df_consortium["ingredient_name"].tolist())

dashboard_rows = []
for _, row in df_risk.iterrows():
    ing         = row["ingredient_name"]
    trust_mult  = _best_trust_mult(ing)
    comp_weight = _best_compliance_weight(ing)
    agnes_score = round(row["total_bom_appearances"] * comp_weight * trus

    # Deterministic savings estimate (higher risk → more to gain from con
    rng = random.Random(hash(ing) % (2 ** 31))
    if row["risk_tier"] in ("CRITICAL", "HIGH"):
        est_pct = round(rng.uniform(10.0, 15.0), 1)
    else:
        est_pct = round(rng.uniform(6.0, 11.0), 1)

    dashboard_rows.append({
        "ingredient_name":  ing,
        "companies":        int(row["company_count"]),
        "bom_appearances":  int(row["total_bom_appearances"]),
        "suppliers":        int(row["supplier_count"]),
        "risk_tier":        row["risk_tier"],
        "vuln_index":       row["vulnerability_index"],
        "trust_score":      int(trust_mult * 100),
```

```
        "compliance_wt":    round(comp_weight, 3),
        "agnes_score":      agnes_score,
        "est_savings":      f"{est_pct}%",
        "gpo_eligible":     "YES" if ing in consortium_set else "–",
    })

df_dashboard = (
    pd.DataFrame(dashboard_rows)
    .sort_values("agnes_score", ascending=False)
    .reset_index(drop=True)
)
df_dashboard.insert(0, "priority", df_dashboard.index + 1)

# ── Summary statistics ──────────────────────────────────────────────
total_boms      = df_dashboard["bom_appearances"].sum()
critical_count  = (df_dashboard["risk_tier"] == "CRITICAL").sum()
gpo_count       = (df_dashboard["gpo_eligible"] == "YES").sum()
avg_savings     = df_dashboard["est_savings"].str.rstrip("%").astype(floa
top_ingredient  = df_dashboard.iloc[0]["ingredient_name"]
top_score       = df_dashboard.iloc[0]["agnes_score"]

SEP = "=" * 75
print(SEP)
print("  AGNES 2.0 — EXECUTIVE DASHBOARD: FULL SUPPLY CHAIN INTELLIGENCE"
print(SEP)
print(f"  Fragmented ingredients analyzed : {len(df_dashboard)}")
print(f"  Total BOM appearances at risk   : {total_boms}")
print(f"  CPG companies involved          : {df_fragmented['company_name'
print(f"  CRITICAL single-source risks    : {critical_count}")
print(f"  GPO-eligible opportunities      : {gpo_count}")
print(f"  Avg estimated savings           : {avg_savings:.1f}%")
print(SEP)
print("\nTop 20 Consolidation Priorities:")
print("(Agnes Score = BOM demand × best compliance weight × best supplier
display(
    df_dashboard.head(20)[[
        "priority", "ingredient_name", "companies", "bom_appearances",
        "suppliers", "risk_tier", "trust_score",
        "agnes_score", "est_savings", "gpo_eligible"
    ]]
)

SEP2 = "–" * 75
print(f"\n{SEP2}")
print("AGNES 2.0 — SYSTEM INTELLIGENCE SUMMARY")
print(SEP2)
print(f"  Agnes has analyzed {len(df_dashboard)} raw ingredient categorie
print(f"  across {df_fragmented['company_name'].nunique()} CPG companies
print(f"\n  KEY FINDINGS:")
print(f"  [RISK]       {critical_count} ingredients have ZERO backup supp
print(f"  [GPO]        {gpo_count} ingredients qualify for group purchasi
print(f"  [TRUST]      Go-Fish scores track 40 suppliers across 20 delive
print(f"  [DISRUPTION] Simulated {DISRUPTED_SUPPLIER} outage: {len(df_dis
print(f"  [PRIORITY]   Top consolidation target: '{top_ingredient}' (Agne
print(f"  [SAVINGS]    Estimated avg. {avg_savings:.1f}% cost reduction a
print(f"\n  Agnes is ready to run deep LLM substitutability analysis")
print(f"  on any of the {len(df_dashboard)} ingredients on demand.")
print(SEP)
```

```
================================================================================
  AGNES 2.0 — EXECUTIVE DASHBOARD: FULL SUPPLY CHAIN INTELLIGENCE
================================================================================
  Fragmented ingredients analyzed : 143
  Total BOM appearances at risk   : 1214
  CPG companies involved          : 60
  CRITICAL single-source risks    : 18
  GPO-eligible opportunities      : 51
  Avg estimated savings           : 9.4%
================================================================================

Top 20 Consolidation Priorities:
(Agnes Score = BOM demand × best compliance weight × best supplier trust)
```

	priority	ingredient_name	companies	bom_appearances	suppliers	risk_tier	trust_s
0	1	vitamin-d3-cholecalciferol	17	33	2	HIGH	
1	2	gelatin	11	30	2	HIGH	
2	3	magnesium-stearate	11	30	2	HIGH	
3	4	microcrystalline-cellulose	13	29	2	HIGH	
4	5	magnesium-oxide	10	25	2	HIGH	
5	6	citric-acid	12	26	2	HIGH	
6	7	silicon-dioxide	10	25	2	HIGH	
7	8	vitamin-c	13	24	2	HIGH	
8	9	stearic-acid	7	24	2	HIGH	
9	10	maltodextrin	8	21	1	CRITICAL	
10	11	whey-protein-isolate	11	20	2	HIGH	
11	12	croscarmellose-sodium	8	19	2	MEDIUM	
12	13	calcium-carbonate	8	20	2	HIGH	
13	14	vitamin-e	10	19	2	MEDIUM	
14	15	glycerin	8	17	1	CRITICAL	
15	16	vitamin-a	10	19	2	MEDIUM	
16	17	whey-protein-concentrate	8	17	2	MEDIUM	
17	18	sunflower-lecithin	7	16	2	MEDIUM	
18	19	biotin	6	18	2	MEDIUM	
19	20	zinc	11	17	2	MEDIUM	

```
──────────────────────────────────────────────────────────────────────────────
─
AGNES 2.0 — SYSTEM INTELLIGENCE SUMMARY
──────────────────────────────────────────────────────────────────────────────
─
  Agnes has analyzed 143 raw ingredient categories
  across 60 CPG companies and 40 suppliers.

  KEY FINDINGS:
  [RISK]       18 ingredients have ZERO backup suppliers — immediate expos
ure
  [GPO]        51 ingredients qualify for group purchasing — instant lever
age
  [TRUST]      Go-Fish scores track 40 suppliers across 20 delivery events
each
  [DISRUPTION] Simulated Prinova USA outage: 135 ingredients affected
  [PRIORITY]   Top consolidation target: 'vitamin-d3-cholecalciferol' (Agn
es score: 79.2)
  [SAVINGS]    Estimated avg. 9.4% cost reduction across all consolidation
s

  Agnes is ready to run deep LLM substitutability analysis
  on any of the 143 ingredients on demand.
==============================================================================
=
```